Provider-Based Stack Detection Engine

A pluggable-provider redesign for DeployGuard's rule-based detection system, modeled on Nixpacks, Cloud Native Buildpacks, and Heroku Buildpacks.

---

1. The Problem

DeployGuard's current detection logic is a single hardcoded chain (e.g. package.json → Node.js → check for "next" → Next.js). As more languages and frameworks are added, this one function has to know about every stack at once, making it harder to extend and easier to break.

2. How Real Platforms Solve This

Nixpacks (Railway), Cloud Native Buildpacks (Heroku, Google Cloud Run, Paketo), and similar tools do not use one big detection function. They use independent **"providers"** — one per language or framework — each of which only answers two questions:

- **Does this repo match me?**
- **If so, what is my install / build / start plan?**

The engine simply loops through registered providers and asks each to self-report a match, instead of one function trying to understand every stack. Cloud Native Buildpacks formalize this further with a dedicated **detect phase**, separate from the build phase, and buildpacks can be grouped so multiple small detectors compose into one profile (e.g. "Node.js + npm + a Procfile" detected together).

3. Recommended Design for DeployGuard

3.1 Provider Interface, Not a Hardcoded Chain

Define one contract every detector implements: `match(repoContext)` returns a boolean or confidence score, and `extract(repoContext)` returns a partial deployment profile. Turn the current chain into separate classes — `NodeProvider`, `PythonProvider`, `RubyProvider`, `NextJsFrameworkDetector`, `DjangoFrameworkDetector` — registered in an array. Adding a new stack (Go, PHP, Java) means writing one new provider class, without touching existing logic.

3.2 Two-Tier Detection: Ecosystem, Then Framework

Make the existing language-then-framework flow explicit as two ordered passes. Pass 1 selects the matching runtime/ecosystem providers; pass 2 runs only the framework detectors relevant to that ecosystem. This keeps detectors composable and avoids a combinatorial explosion of rules.

3.3 Confidence Scoring Instead of First-Match-Wins

Rather than pass/fail, give each provider a lightweight confidence score — e.g. package.json + next.config.js + a "next" dependency scores high confidence "Next.js"; package.json alone scores medium

confidence "generic Node.js". Select the highest-scoring match, and if it falls below a threshold, fall back to the generic language template, consistent with DeployGuard's existing fallback to user-provided custom Dockerfiles.

3.4 Monorepo / Multi-Service Support

This is the biggest real-world gap in a single-repo detector. Add a low-cost first step: scan for subdirectories with their own manifest files (e.g. `/frontend/package.json`, `/backend/requirements.txt`) and let the user select which subpath to deploy, or run detection per service for multi-container projects. Even just flagging "this looks like a monorepo, please select a root directory" is a significant flexibility win for low engineering cost.

3.5 User-Overridable Plan, Not Just a User-Provided Dockerfile

Nixpacks lets users override individual pieces of an auto-generated plan through a config file, including which providers contribute to the build and specific build/start commands. DeployGuard should let users override individual detected fields (build command, port, health-check path) directly in the Pre-Flight Validation screen, instead of forcing an all-or-nothing choice between "fully auto-detected" and "fully custom Dockerfile." This is far cheaper than writing more detection rules and fits naturally into the existing Pre-Flight Validation page.

4. Design Advantage: Version Pinning

Nixpacks-style detection is genuinely weaker on exact version control, since Nixpacks resolves packages against a Nix package-set snapshot rather than pinning an exact version. Because DeployGuard already generates a real Dockerfile per detected template rather than a Nix environment, it does not inherit this weakness — worth stating explicitly as a design advantage in the proposal's literature review / comparison table.

Approach	Detection Model	Version Pinning
Nixpacks	Ordered providers, pass/fail	Weak (Nix snapshot-based)
Cloud Native Buildpacks	Grouped detect-phase buildpacks	Depends on buildpack
DeployGuard (proposed)	Scored, composable providers	Strong (generates real Dockerfile)

5. Proposed Detection Flow

```
Repo
-> Language Providers run in parallel, score confidence
-> Highest-confidence ecosystem selected
-> Framework Providers for that ecosystem run
-> Highest-confidence framework selected (or "generic" fallback)
-> Monorepo check: multiple manifests found? -> prompt for subpath / multi-service mode
-> Deployment Profile assembled from provider outputs
```

```
-> User can override individual fields in Pre-Flight screen before committing
-> If profile confidence too low anywhere -> require custom Dockerfile
```

6. Summary

This design keeps DeployGuard fully rule-based and deterministic, with no reliance on AI inference or hallucination risk, while making "add support for a new stack" a matter of writing one new provider class rather than editing a monolithic detection function. It also provides a concrete, defensible answer to the question: *"What happens with a repo you didn't anticipate?"* — the system falls back gracefully to a generic template or a user-provided Dockerfile, with confidence scoring making that fallback threshold explicit.